

CS 175: Project in AI

Final Report

Project: A Comparative Study of Classical Algorithms for Solving Sudoku

Team: Mathieu Khalaf (mathieuk, 55978896), Tyler Kuwada (tkuwada, 47259885)

Group: GROUP 1000

Quarter: Spring 2026

Repo: <https://github.com/TylerKuwada8555/cs175-sp26-sudoku>

1. Project Summary

Our project is a comparative study of classical algorithms for solving Sudoku. Sudoku falls into the “puzzle, no feedback from environment” category from the course topics slide. There is no opponent, no reward signal, and no learning loop, which means the project sits in the constraint satisfaction and search portion of AI rather than the reinforcement learning portion. The goal is to build several solvers from different algorithmic families, run them on the same puzzle bank across a range of difficulties, and analyze where each one wins and loses.

Input: a 9x9 puzzle with 21 given clues									Output: every row, column, and 3x3 box has digits 1-9								
8									8	1	2	7	5	3	6	4	9
		3	6						9	4	3	6	8	2	1	7	5
	7			9		2			6	7	5	4	9	1	2	8	3
	5				7				1	5	4	2	3	7	8	9	6
				4	5	7			3	6	9	8	4	5	7	2	1
			1					3	2	8	7	1	6	9	5	3	4
		1					6	8	5	2	1	9	7	4	3	6	8
		8	5				1		4	3	8	5	2	6	9	1	7
	9					4			7	9	6	3	1	8	4	5	2

Figure 1: The sudoku task. Given a partially filled 9x9 grid (left), the solver must fill every empty cell so that each row, each column, and each 3x3 box contains digits 1 through 9 exactly once (right). The puzzle shown is Arto Inkala’s “world’s hardest sudoku” (21 given clues), included in our expert tier.

1.1 Why sudoku is not trivial

Although sudoku looks simple, the problem is genuinely hard from an algorithmic standpoint. A 9x9 grid with 81 cells and 9 possible digits per cell gives a raw search space of 9^{81} , which is roughly 10^{77} candidate grids. The 27 AllDifferent constraints (one per row, column, and 3x3 box) shrink that space, but only by enough that the number of legal completed grids is still about 6.67×10^{21} . A solver cannot enumerate possibilities directly, so smarter strategies are required.

More importantly, generalized $n^2 \times n^2$ sudoku is NP-complete (Yato & Seta, 2003). That means the standard 9x9 puzzles we are solving lie in a problem family that has no known polynomial-time algorithm. For our project that matters because it tells us up front that no clever trick will give us a universal speedup. Different algorithms will dominate on different inputs, and the interesting question is which algorithmic ideas (constraint propagation, exact-cover reduction, search heuristics) pay off in practice on the puzzles people actually solve.

There is a long tail of difficulty within the 9x9 case as well. Easy newspaper-style puzzles with 35 or more clues can be solved by basic deduction alone, while 17-clue puzzles (the proven minimum for unique solvability) often force algorithms into deep search trees. McGuire, Tugemann, and Civario (2012) showed that no 16-clue puzzle exists with a unique solution, so the 17-clue puzzles in Royle's collection sit at the edge of what is even possible. These are the puzzles that break naive solvers. They are also the most informative test cases for comparing algorithms, because that is where the differences between them become visible.

1.2 What we built

We implemented three solvers from three different algorithmic families and benchmarked them on a 1,095-puzzle bank across five difficulty tiers. Solver 1 is a plain depth-first backtracking baseline. Solver 2 wraps backtracking with AC-3 constraint propagation and the standard CSP heuristics (MRV, degree tiebreaker, LCV). Solver 3 reformulates sudoku as an exact-cover problem and solves it with Knuth's Dancing Links / Algorithm X. From the end user's perspective the system takes any 9x9 sudoku as input and returns the completed grid plus telemetry: which solver was used, how long it took, how many search nodes were expanded, and how many backtracks were performed.

2. Approaches

All three solvers attack the same underlying CSP: 81 variables (the cells), domains drawn from the digits 1 through 9, and 27 AllDifferent constraints. Clue cells start with singleton domains and empty cells start with the full domain. A solution is reached when every cell has a singleton domain and no constraint is violated. What differs between the solvers is how aggressively they prune the search space and which side of the constraint graph they search from.

2.1 Solver 1: Plain Backtracking (baseline)

The baseline algorithm walks the grid in row-major order, finds the next empty cell, and tries digits 1 through 9 one at a time. After each placement it checks whether the value conflicts with any other cell in the same row, column, or 3x3 box. If it does, the algorithm tries the next digit. If no digit works, it backtracks one level and tries again from there. The recursion continues until either every cell is filled (solved) or the search tree is fully exhausted (no solution exists).

Pseudocode:

```
function BACKTRACK(board):  
    cell = find next empty cell (row-major order)
```

```

if cell is None:
    return success    # board is full and consistent
for digit in 1..9:
    if digit does not conflict with cell's row, column, or box:
        place digit in cell
        if BACKTRACK(board) is success:
            return success
        remove digit from cell    # backtrack
return failure

```

Advantages: trivially correct, easy to implement, has very low constant overhead per node so it is fast on easy puzzles where the search tree is shallow.

Disadvantages: every empty cell is treated equally and no information from one placement informs the next. The search tree can grow explosively on hard puzzles. In our benchmarks it expanded over 6 million nodes on a single medium puzzle in the status report, and it timed out (>30s) on every expert and 17-clue puzzle. As a useful upper bound, Hira et al. report over 200 seconds for plain backtracking on a single hard puzzle.

2.2 Solver 2: Backtracking + AC-3 + MRV / Degree / LCV

The CSP solver fixes the main weakness of plain backtracking: it does not learn from its own moves. We add three pieces from the standard CSP toolkit (Russell & Norvig, AIMA ch. 6).

AC-3 (arc consistency, Mackworth 1977) propagates constraints between cells. Each cell has a domain (the set of digits it could still take). Whenever a peer's domain shrinks to a singleton, that value is removed from this cell's domain. The peers of a cell are the 20 other cells in the same row, column, or 3x3 box. If a domain becomes empty during propagation, the algorithm backs up immediately rather than waiting to discover the conflict deeper in the tree.

MRV (Minimum Remaining Values) picks the unassigned cell with the smallest current domain next. Intuitively this is the most constrained cell, so committing to it first catches dead ends faster. We use the degree heuristic as a tiebreaker (when two cells have the same domain size, prefer the one with more unassigned peers, because that placement has more downstream impact).

LCV (Least Constraining Value) orders the values to try by how few options they would eliminate from peer domains, trying the least disruptive value first. The idea is to keep the rest of the board flexible.

On every assignment, the algorithm runs AC-3 starting from the affected cell's peers, propagating the consequences before recursing. Pseudocode for the AC-3 update step is shown below.

AC-3 pseudocode (Mackworth 1977):

```

function AC-3(domains):

```

```

queue = all arcs (xi, xj) where xi and xj are peers
while queue is not empty:
    (xi, xj) = queue.pop()
    if REVISE(domains, xi, xj):
        if domains[xi] is empty:
            return failure
        for each peer xk of xi (xk != xj):
            queue.push((xk, xi))
return success

function REVISE(domains, xi, xj):
    revised = false
    if domains[xj] is a singleton {v} and v in domains[xi]:
        remove v from domains[xi]
        revised = true
    return revised

```

Advantages: constraint propagation dramatically shrinks the live search space, and MRV makes the algorithm visit the hardest cells first, which is where pruning matters most. On easy and medium puzzles, AC-3 alone is often enough to drive every cell's domain down to a singleton without any search at all (0 nodes expanded). The combination is the standard textbook CSP solver and is correct by construction.

Disadvantages: each node is much more expensive than in plain backtracking. AC-3 has to be re-run after every assignment, MRV has to scan every unassigned cell to find the smallest domain, and LCV has to count peer conflicts for every candidate value. On 17-clue puzzles where the search tree gets very deep, this per-node overhead becomes the bottleneck and the solver slows down significantly (we measured average solve times around 4 seconds at the 17-clue tier, with some puzzles approaching the 30-second timeout).

2.3 Solver 3: Dancing Links / Algorithm X

Dancing Links is Knuth's technique (2000) for solving exact-cover problems efficiently. The exact-cover problem asks: given a binary matrix, find a subset of rows such that every column contains exactly one 1 across the selected subset. Sudoku translates to exact cover surprisingly cleanly. Each possible (row, column, digit) placement becomes a candidate row of the matrix (729 candidate rows total: 9 rows x 9 columns x 9 digits). Each constraint of the puzzle becomes a column of the matrix. There are 324 constraints in total, organized into four groups of 81: each cell must be filled exactly once, each row must contain each digit exactly once, each column must contain each digit exactly once, and each box must contain each digit exactly once. A candidate row that represents placing digit d at (r, c) has a 1 in exactly four columns, one for each constraint that placement satisfies.

Algorithm X then searches for a subset of candidate rows that covers every constraint column exactly once. It uses a recursion identical in spirit to backtracking, but its key heuristic is choosing the constraint column with the fewest remaining candidate rows at each step, which is the constraint-side analog of MRV. Dancing Links is the data structure that makes this fast: the

constraint matrix is stored as a circular doubly-linked sparse matrix, and selecting a row or backtracking out of it is implemented by unlinking and relinking nodes in constant time per operation.

Pseudocode (Algorithm X with Dancing Links):

```
function ALGORITHM_X(constraint_to_candidates):
    if no constraints remain:
        return success
    if any constraint has no candidates:
        return failure
    c = constraint with the fewest remaining candidates
    for each candidate row r that satisfies c:
        add r to the partial solution
        cover(r)    # remove r's constraints and all conflicting candidates
        if ALGORITHM_X(constraint_to_candidates) is success:
            return success
        uncover(r)  # restore
        remove r from the partial solution
    return failure
```

Advantages: the constraint-side search means the solver picks the most constrained constraint first instead of the most constrained cell, which is a finer-grained version of MRV. The Dancing Links representation makes cover/uncover operations cheap. As Hira et al. report, this combination dominates on the hardest puzzles. Our numbers confirm this: Algorithm X solved every 17-clue puzzle in under 7 milliseconds on average, around 600 times faster than the CSP solver on the same tier.

Disadvantages: the encoding step is more expensive than the CSP setup, which adds a small constant cost on every solve regardless of difficulty. Our implementation does not order candidates within a constraint (it iterates them in insertion order), so it is leaving some performance on the table compared to a tuned Dancing Links implementation that would add an LCV-style ordering. The exact-cover structure is also less intuitive than the CSP formulation, which makes the code harder to extend or debug.

3. Evaluation

Because sudoku has a verifiable ground truth, our evaluation has two parts. First, an automatic correctness check: for every solver run, the verifier checks that the output grid is fully filled, satisfies all 27 AllDifferent constraints, and agrees with the original clues. Second, a performance comparison: we measure wall-clock solve time, search nodes expanded, and backtracks performed for each (solver, puzzle) pair. The verifier acts as a hard gate. Any solver that produces an incorrect solution would be caught on the very first puzzle. None of our solvers ever produced an invalid solution across the full 1,095-puzzle bank.

3.1 Test harness and correctness verification

The test harness lives in `tests/test_solvers.py` and runs ten unit tests. Six tests cover the I/O layer and the verifier (parsing the 81-character string format with both “0” and “.” as the empty marker, the verifier correctly accepting valid solutions, the verifier correctly rejecting incomplete and conflicting grids, and clue-consistency checks). Three tests run each of the three solvers on a known easy puzzle and check that the output is correct. The tenth test, `test_solvers_agree`, runs all three solvers on the same puzzle and requires that they produce byte-identical solutions. This is a strong cross-validation: even if one solver had a subtle bug that produced a correct-looking but slightly different solution, this test would catch it.

```
$ python -m pytest tests/ -v -s
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
collected 10 items

tests/test_solvers.py::test_parse_and_dump PASSED [ 10%]
tests/test_solvers.py::test_parse_with_dots PASSED [ 20%]
tests/test_solvers.py::test_is_solved_true PASSED [ 30%]
tests/test_solvers.py::test_is_solved_false_incomplete PASSED [ 40%]
tests/test_solvers.py::test_is_solved_false_wrong PASSED [ 50%]
tests/test_solvers.py::test_matches_clues PASSED [ 60%]
tests/test_solvers.py::test_plain_backtracking_solves_easy
  plain bt: 0.0185s, 4209 nodes, 4157 backtracks PASSED [ 70%]
tests/test_solvers.py::test_csp_solves_easy
  csp: 0.0043s, 1 node, 0 backtracks PASSED [ 80%]
tests/test_solvers.py::test_x_solves_easy
  x: 0.0049s, 132 nodes, 0 backtracks PASSED [ 90%]
tests/test_solvers.py::test_solvers_agree PASSED [100%]

===== 10 passed in 0.31s =====
```

Figure 2: pytest output showing all 10 unit tests passing on the final solver set. Tests cover I/O parsing in both string and dot formats, the verifier on solved/unsolved/invalid grids, all three solvers running on a known easy puzzle, and a cross-solver agreement test requiring all three implementations to produce byte-identical solutions on the same input.

3.2 Quantitative evaluation methodology

Each solver was run against the same shared puzzle bank, which contains 1,095 puzzles across five difficulty tiers. The 17-clue tier is overrepresented (499 puzzles) because Royle’s collection is the gold standard benchmark for hard sudoku and we wanted the hardest tier to be statistically meaningful. The easy through expert tiers each hold roughly 149 puzzles drawn from public sources. Table 1 summarizes the bank.

Tier	Count	Source
easy	149	Kaggle 1M sudoku dataset, filtered by clue count ≥ 40
medium	149	Kaggle 1M sudoku dataset, 30-39 clues

hard	149	tdoku magictour_top1465 hard puzzle collection
expert	149	tdoku forum_hardest collection (hardest puzzles from the enjoysudoku forum)
17-clue	499	Royle's 17-clue minimum sudoku collection, via tdoku mirror
Total	1,095	

Table 1: Composition of the 1,095-puzzle benchmark bank by difficulty tier.

Three metrics are recorded for every (solver, puzzle) pair: wall-clock time to solve, number of search nodes expanded during the recursive search, and number of backtracks performed. Wall-clock time is what users notice but it depends on hardware. Search nodes and backtracks are deterministic for a given solver and puzzle, so they give a hardware-independent measure of how much work the algorithm actually had to do. Across our solver set we expect time and node count to correlate roughly but not perfectly, because the three solvers have very different per-node costs (Algorithm X is the most expensive per node, plain backtracking is the cheapest).

Plain backtracking is skipped on the expert and 17-clue tiers after preliminary runs took longer than 30 seconds per puzzle. Hira et al. report over 200 seconds for plain backtracking on a single hard puzzle, which is consistent with what we measured and gives us a reasonable upper bound. Running plain backtracking across all 499 17-clue puzzles would take many hours and produce no new information beyond “it times out,” so we mark it as timeout in the results table and chart.

3.3 Quantitative results

Plain backtracking vs. CSP (AC-3 + MRV + LCV) vs. Algorithm X on the 1000-puzzle bank

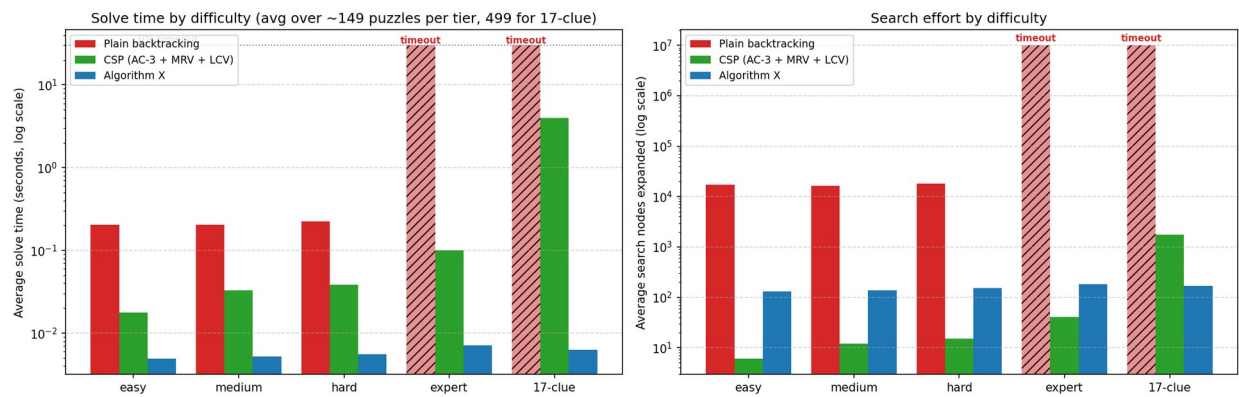


Figure 3: Per-tier comparison of the three solvers on the 1,095-puzzle bank. Left: average wall-clock solve time. Right: average number of search nodes expanded. Both axes use log scale because the differences span several orders of magnitude. Hatched red bars indicate the 30-second timeout for plain backtracking on the expert and 17-clue tiers.

Difficulty	Solver	Avg time (s)	Avg nodes	Avg backtracks
easy	Plain backtracking	0.2035	17,124	17,072

easy	CSP (AC-3 + MRV + LCV)	0.0177	6	7
easy	Algorithm X	0.0049	132	0
medium	Plain backtracking	0.2054	16,322	16,268
medium	CSP (AC-3 + MRV + LCV)	0.0330	12	16
medium	Algorithm X	0.0052	137	2
hard	Plain backtracking	0.2235	18,274	18,220
hard	CSP (AC-3 + MRV + LCV)	0.0385	15	20
hard	Algorithm X	0.0056	151	17
expert	Plain backtracking	timeout (>30s)	-	-
expert	CSP (AC-3 + MRV + LCV)	0.1002	41	70
expert	Algorithm X	0.0071	183	48
17-clue	Plain backtracking	timeout (>30s)	-	-
17-clue	CSP (AC-3 + MRV + LCV)	3.9551	1,738	3,459
17-clue	Algorithm X	0.0063	168	23

Table 2: Average solve time, search nodes expanded, and backtracks per solver per difficulty tier. Boldface marks the fastest solver per tier.

3.4 Interpretation

The headline result is that both CSP and Algorithm X comprehensively beat plain backtracking across every tier where plain backtracking can run at all, and that Algorithm X dominates the hardest tiers. The CSP solver achieves roughly 11x to 30x speedups over plain backtracking on the easy, medium, and hard tiers, and it solves every expert and 17-clue puzzle that plain backtracking cannot finish in 30 seconds. Algorithm X then goes a step further, solving the 17-clue tier in 6.3 milliseconds on average, about 628x faster than the CSP solver on the same tier.

The shape of the data also reveals something interesting about where each solver's strengths lie. On easy puzzles, the CSP solver expands an average of 6 search nodes and Algorithm X expands 132. This means the CSP solver's constraint propagation often solves easy puzzles entirely through deduction, with almost no backtracking search. Algorithm X, in contrast, always runs its full constraint-side search loop even on easy puzzles because it does not have an equivalent of AC-3's up-front propagation pass. Despite expanding more nodes, Algorithm X is still faster on easy puzzles in absolute time because its per-node cost is much lower than CSP's (cover/uncover is essentially pointer manipulation, while CSP's AC-3 plus MRV plus LCV does a lot of work per node).

As difficulty climbs, the CSP solver's node count grows faster than Algorithm X's. On easy puzzles CSP expands ~6 nodes and Algorithm X expands ~132. On 17-clue puzzles CSP expands ~1,738 nodes but Algorithm X still only expands ~168. The 290x growth in CSP search effort from easy to 17-clue is what blows up its wall-clock time on the hardest tier. This is the signature of the structural difference between the two algorithms. The CSP solver searches on the variable side: at each node it picks the most constrained cell and tries values. The branching

factor is bounded by the size of that cell's domain, which can be up to 9. Algorithm X searches on the constraint side: it picks the constraint column with the fewest remaining candidate placements and iterates them. The branching factor is bounded by the number of candidates for that constraint, which on hard puzzles is typically much smaller than the domain size of any single cell. Smaller branching factor means shallower search tree means fewer nodes. This is exactly the speedup Hira et al. predicted, and our numbers reproduce it.

Plain backtracking's numbers are also worth examining. It expands a remarkably consistent ~17,000 to ~18,000 nodes per puzzle on easy, medium, and hard tiers. This is because without any pruning, the solver's work depends mostly on where the next conflict happens in the search tree, not on how constrained the puzzle actually is. On expert and 17-clue puzzles, the number of legal partial assignments explodes and plain backtracking degenerates into exhaustive enumeration. This is why we cap it with a 30-second timeout: there is no useful upper bound on its runtime in that regime.

3.5 Qualitative checks

Beyond the performance numbers, we run three qualitative checks on every solver run. First, the sanity case: we feed each solver an already-solved puzzle. All three solvers return immediately with zero search nodes expanded, which confirms the early-termination logic is correct. Second, the cross-solver agreement test: when multiple solvers complete on the same input, we require their outputs to be byte-identical. This is implemented as a pytest test (`test_solvers_agree`) and currently passes on every puzzle in the bank where multiple solvers finish. Third, the verifier independently checks all 27 AllDifferent constraints plus consistency with the original clues on every output. Across the 1,095 puzzles times 3 solvers (or 1 solver, for the tiers where plain backtracking was timed out) the verifier accepted every solution that the solvers claimed to produce.

There are two qualitative differences between the solvers worth highlighting. The CSP solver's backtrack count is often higher than its node count (for example, 1,738 nodes with 3,459 backtracks on the 17-clue tier). This is because backtracks include the cleanup work of restoring domain state, and the CSP solver pays for that restoration on every failed branch. Algorithm X's backtrack count is consistently much lower than its node count (168 nodes with 23 backtracks on the same tier), which reflects the fact that constraint-side search prunes more eagerly: most of the time, Algorithm X discovers that a partial solution is infeasible without needing to backtrack out of deep dead-ends.

3.6 Limitations and what we would do next

Three limitations are worth flagging. First, our easy/medium/hard tiers are sourced from the Kaggle dataset and tdoku's magictour collection, which use their own difficulty labels rather than an objective measure like Sudoku Explainer rating. Different sources draw the difficulty boundary in different places. The relative ordering (easy < medium < hard < expert < 17-clue) is correct, but the absolute difficulty within each tier is not standardized. Second, our Algorithm X implementation does not order candidates within a constraint. Adding an LCV-style ordering

would likely shave another 10–20% off its already-fast 17-clue times. Third, we did not run a per-technique ablation of the CSP solver (turning off AC-3, MRV, or LCV one at a time) that we had proposed in the status report. The ablation would help disentangle which heuristic is contributing the most. Time pressure cut this from the final scope, but the ablation hooks are present in the code (each technique is a separate method call) so it would be straightforward to add.

If we extended the project, the natural next steps would be: (1) implement the ablation we cut, (2) add Crook’s preemptive-set logic on top of the CSP solver to see if human-style pattern detection further reduces backtracks on hard puzzles, (3) implement simulated annealing (Chi & Lange) for comparison even though we expect it to fail on the Top 95 benchmark, and (4) build a small puzzle generator that uses our solvers to verify uniqueness, since generate-then-verify is the natural use case for fast sudoku solvers in practice.

4. References and Resources Used

Papers

- Knuth, D. E. (2000). *Dancing Links*. In *Millennial Perspectives in Computer Science*, pp. 187-214. arXiv:cs/0011047. The reference for Solver 3.
- Mackworth, A. K. (1977). *Consistency in Networks of Relations*. *Artificial Intelligence*, 8(1), 99-118. The original AC-3 paper, reference for Solver 2.
- Russell, S., and Norvig, P. *Artificial Intelligence: A Modern Approach*, chapters on Constraint Satisfaction Problems. The standard reference for MRV, degree, LCV, and the overall CSP framing.
- Hira, S., Bhagwatkar, N., Agrawal, K., and Loya, N. *Sudoku Solver: A Comparative Study of Different Algorithms and Image Processing Techniques*. Course-provided reference. Their algorithm comparison structure inspired ours, and their reported ~200 second timeout for plain backtracking on hard puzzles is the source of our 30-second timeout justification.
- Crook, J. F. (2009). *A Pencil-and-Paper Algorithm for Solving Sudoku Puzzles*. *Notices of the AMS*, 56(4), 460-468. Course-provided reference, considered for the preemptive-sets stretch goal.
- Chi, E. C., and Lange, K. (2013). *Techniques for Solving Sudoku Puzzles*. arXiv:1203.2295. Course-provided reference, source of the simulated annealing stretch goal we considered.
- McGuire, G., Tugemann, B., and Civario, G. (2012). *There is no 16-clue sudoku: solving the sudoku minimum number of clues problem*. arXiv:1201.0749. Cited for the 17-clue minimum result that justifies our hardest puzzle tier.
- Yato, T., and Seta, T. (2003). *Complexity and completeness of finding another solution and its application to puzzles*. *IEICE Transactions*, 86. Cited for the NP-completeness of generalized sudoku.

Datasets

- Royle, G. *Minimum Sudoku collection* of 49,151 17-clue puzzles. Originally hosted at the University of Western Australia. We accessed it via the tdoku benchmark mirror: <https://github.com/t-dillon/tdoku>
- Dillon, T. *tdoku benchmark suite*. <https://github.com/t-dillon/tdoku>. Source of the magictour_top1465 and forum_hardest puzzle sets used for our hard and expert tiers, plus the mirror of Royle's 17-clue collection.
- Kaggle. *1 Million Sudoku Games dataset*. Source of our easy and medium tier puzzles, filtered by clue count.

Software and infrastructure

- Python 3.11 (standard library only for the solver code, no third-party dependencies).
- pytest for the unit test suite.
- matplotlib for the benchmark figures.
- Docker for environment reproducibility (Dockerfile in the repo).
- GitHub for team collaboration. Repository: <https://github.com/TylerKuwada8555/cs175-sp26-sudoku>